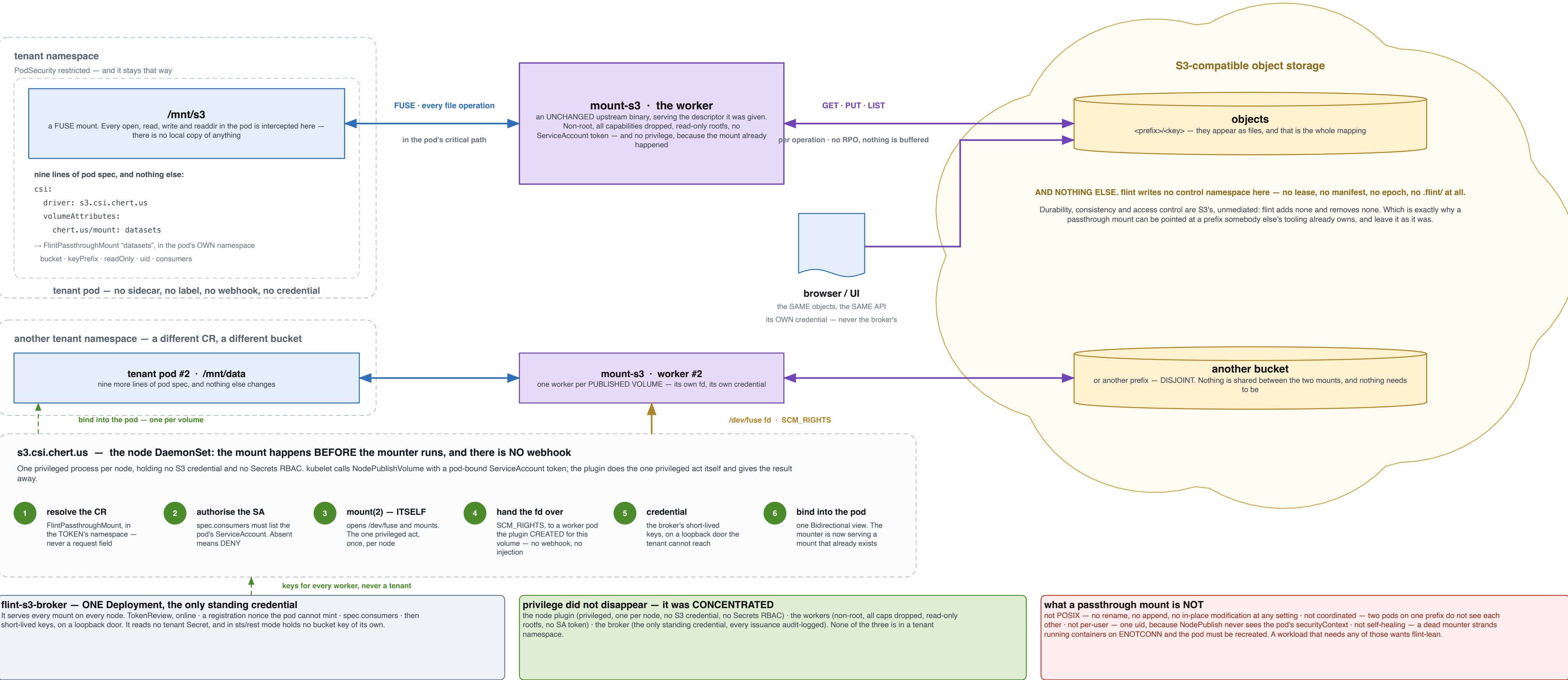


Data flow

Every file operation in the pod is an S3 request. Nothing is buffered, flint writes nothing of its own into the bucket — and the privilege that makes the mount is not in the tenant's namespace.



THE MOUNT HAPPENS BEFORE THE MOUNTER RUNS, and that is the whole trick. The node plugin opens /dev/fuse and calls mount(2) itself — the one act that genuinely needs privilege, done once, per node — then hands the descriptor to an unprivileged worker pod over SCM_RIGHTS, where an unchanged mount-s3 serves it needing no privilege at all. The tenant pod is given nothing: no sidecar, no label, no webhook, no credential, no privileged container. It declares a csi: volume naming a CR in its own namespace and stays admissible under PodSecurity restricted.

THERE IS NO CONTROLLER AND NO STATUS, because nothing about a passthrough mount converges. The CRD is the whole control surface and the node plugin reads it directly; what a (cluster issuer, namespace, ServiceAccount, CR) tuple is actually GIVEN is decided by the broker's backend — static for a rig, sts for a proxy that speaks AssumeRoleWithWebIdentity, rest for an application's own JWT-enforcing API. A CR on a cluster the project's policy never named yields nothing.

A UI NEEDS NOTHING BUILT, which is the one place passthrough is the cheapest of the four. The bucket IS the API: a UI lists and GETs the same keys the mount presents, so there is no gateway to run, no inbox to drain and no manifest to respect. What it costs is the same thing the mount costs — no coordination. Two S3 clients with no lease between them can overwrite each other, and Mountpoint's own limits still bind what a UI can usefully do: no rename, no append, no in-place modification, so "edit a file" is always a whole-object PUT.

EVERY OPERATION IS A REQUEST, so there is no recovery point to state: nothing is buffered and nothing is flushed. That is also the cost — the S3 round trip is in the pod's critical path, where lean has local disk and lite has one wire to one process. The bucket, meanwhile, is untouched by flint: this is the only front end you can point at a prefix someone else's tooling owns.

A WORKER IS NEVER TAKEN AWAY FROM A TENANT STILL USING ITS MOUNT — by ORDERING, not by a PodDisruptionBudget. A budget only applies to the eviction API, and on a spot fleet the ordinary case is kubelet's graceful shutdown, where the eviction API is not involved at all. What is built instead is a PriorityClass that ranks a worker above its tenants, so kubelet tears it down last, and a preStop hook that waits for NodeUnpublishVolume to release the volume.

